

Paradigmi di programmazione

Contents

1	Introduzione	4
2	Descrizione dei paradigmi	5
2.1	Paradigma imperativo	5
2.2	Paradigma object-oriented	5
2.3	Paradigma funzionale	5
2.4	Programmazione concorrente	5
2.5	Programmazione parallela	5
3	Il ruolo chiave dei tipi	6
4	Linguaggi	7
4.1	Fortran	7
4.2	ALGOL 60	7
4.3	LISP	7
4.4	SIMULA 67	7
4.5	C	8
4.6	Pascal	8
4.7	Smalltalk	8
4.8	PROLOG	8
4.9	C++	8
4.10	Java	8
4.11	Altri linguaggi popolari	9
4.12	Come crescono i linguaggi?	9
5	Approccio scientifico	10
6	Lambda Calculus	11
6.1	Definizione	11
6.2	Lambda espressioni	11
6.2.1	Segnatura e funzione identità	11
6.2.2	Applicazione	11
6.2.3	Funzione costante	11
6.2.4	Funzioni annidate	12
6.2.5	Ordine di semplificazione	12
6.2.6	Funzioni come parametro	12
6.2.7	Prolungamento di espressione	12
6.2.8	Parametri attuali numerici	13
6.2.9	Ambiguità	13
6.3	Convenzioni sintattiche	13
6.4	Variabili libere e legate	13
6.5	Definizione sintattica	14
6.6	Alberi per lambda espressioni	14
6.7	Alpha equivalenza	14
6.8	Valutazione di lambda espressioni	14
6.9	Beta reduction e beta equivalenza	15

6.10	Non terminazione	16
6.11	Forma normale	16
6.12	Teorema di Church Rosser	16
6.13	Funzioni di ordine superiore e currying	16
6.14	Valutazione dell'applicazione di funzione	17
6.15	Implementazione della ricorsione	17
6.16	Variabili booleane	18
6.17	Condizionale	19
6.18	Numeri	19
7	Regole di inferenza	20
7.1	Sintassi	20
7.2	Successore	20
7.3	Predecessore	20
7.4	IsZero	20
7.5	If-Else	20
8	Controllo dei tipi	21
8.1	Tipi	21
8.2	Well-typed e ill-typed	21
8.3	Type Safety: Correttezza	21
8.4	Controllo su successore	21
8.5	Controllo su predecessore	21
8.6	Controllo su isZero	21
8.7	Controllo su if-else	22
9	Lambda Calculus Tipato	23
9.1	L'introduzione dei tipi	23
9.2	Sistema di controllo dei tipi	23
9.3	Sintassi dei tipi	24
9.4	Ambiente di tipo	24
9.5	Substitution Lemma	25
9.6	Dichiarazioni locali	25
9.7	Ricorsione	25
10	OCaml	26
10.1	ML, un antenato	26
10.2	Come mai NON studiare OCaml (Ma invece si)	26
10.3	Esecutore di OCaml	26
11	Esercizi	27
11.1	Esercizio 1	27
11.2	Esercizio 2	27
11.3	Esercizio 3	27
11.4	Esercizio 4	28
11.5	Esercizio 5	28
11.6	Esercizio 6	28

1 Introduzione

Email Docente: chiara.bodei@unipi.it

E-Learning: <https://elearning.di.unipi.it/course/view.php?id=1167> Il Teams è privato, non si può entrare. (Future notice)

Lo scopo del corso è di facilitare l'apprendimento di nuovi linguaggi di programmazione attraverso lo studio dei paradigmi.

2 Descrizione dei paradigmi

Non esiste un paradigma migliore degli altri: la scelta dipende dalla situazione e dal problema da risolvere.

Molti programmi possono utilizzare, per sottoproblemi diversi, approcci differenti (**multiparadigma**).

2.1 Paradigma imperativo

Il **paradigma imperativo** descrive il programma come una sequenza di istruzioni che modificano lo stato del programma.

Il programmatore specifica **come** ottenere il risultato, indicando esplicitamente le operazioni da eseguire.

Esempi di costrutti tipici sono assegnamenti, cicli e istruzioni condizionali.

2.2 Paradigma object-oriented

Il **paradigma object-oriented** (o **a oggetti**) organizza il programma attorno a oggetti che combinano **dati** e **comportamenti**.

Gli oggetti appartengono a **classi**, che ne definiscono caratteristiche e operazioni.

I principali concetti sono **incapsulamento**, **ereditarietà**, **polimorfismo** e **astrazione**.

2.3 Paradigma funzionale

Il **paradigma funzionale** considera il programma come una composizione di **funzioni**, cercando di evitare modifiche dello stato e effetti collaterali.

Le funzioni possono essere trattate come valori, passate come argomenti e restituite come risultati.

Sono importanti concetti come **immutabilità**, **funzioni pure**, **ricorsione** e **funzioni di ordine superiore**.

2.4 Programmazione concorrente

La **programmazione concorrente** permette a più attività di essere **in esecuzione nello stesso intervallo di tempo**, anche se non vengono necessariamente eseguite simultaneamente.

Le attività possono alternarsi e comunicare o sincronizzarsi tra loro.

È utile, ad esempio, per gestire contemporaneamente richieste di utenti, operazioni di I/O o attività indipendenti.

2.5 Programmazione parallela

La **programmazione parallela** consiste nell'eseguire più operazioni **simultaneamente**, generalmente sfruttando più processori o core.

L'obiettivo principale è ridurre il tempo necessario per completare un'elaborazione, dividendo il problema in parti che possono essere eseguite in parallelo.

La programmazione parallela è quindi particolarmente utile per problemi scomponibili in attività indipendenti o parzialmente indipendenti.

3 Il ruolo chiave dei tipi

I **tipi** nella programmazione sono molto importanti, in quanto dichiarano gli intenti di dati e funzioni.

Il **Type Checking** (presente per esempio in TypeScript) assicura il corretto svolgimento delle operazioni.

Inoltre, il type checking rende il programma più efficiente ed evita errori nelle operazioni. Si dice che il type checking offra una **correttezza parziale** (solo per alcuni tipi di errori) ed è un esempio di tecnica di analisi di programma (statica e dinamica).

4 Linguaggi

4.1 Fortran

Fortran (FORMula TRANsfer) è uno dei linguaggi più antichi, realizzato da **John Backus** (1924-2007) negli anni 50 all'IBM, primo linguaggio di alto livello implementato efficacemente (memoria statica, senza annidamento).

Ha introdotto le variabili come le conosciamo oggi, i cicli, le procedure, le etichette delle istruzioni e molto altro.

Le prime versioni non supportavano la ricorsione e includevano caratteristiche oggi abbandonate.

4.2 ALGOL 60

ALGOL (ALGOritmic Language) fu progettato nel 1958-1960 come linguaggio universale.

Ha introdotto innovazioni fondamentali:

- Sintassi formalmente specificata (BNF)
- Scoping lessicale / blocchi
- Procedure modulari
- Passaggi per nome
- Procedure ricorsive
- Dichiarazioni di tipi di variabili
- Allocazione dello stack e record di attivazione

4.3 LISP

Il **LISP** (LISt Processor) è un linguaggio funzionale, sviluppato da **John McCarty** nel 1958 al MIT per la manipolazione di liste.

Precursore di molti concetti, tra cui la gestione dinamica della memoria (heap e garbage collector).

4.4 SIMULA 67

SIMULA 67 è stato sviluppato da **Ole-Johan** e **Kristen Nygaard**, è un'estensione di ALGOL 60.

Fu il primo linguaggio orientato agli oggetti, con classi, oggetti, ereditarietà e sottoclassi. Introdusse anche passaggi per valore e riferimento, puntatori e **co-routine** (simili ai thread).

4.5 C

Introdotta nei Bell Labs 1972 da **Ritchie**, il suo sviluppo è strettamente legato a **UNIX** e ai sistemi operativi.

Maggiori possibilità di accessi alle funzionalità di basso livello della macchina.

Gestione manuale della memoria (puntatori ma no garbage collection). Le funzioni non si possono annidare.

Controllo dei tipi non forte.

Linguaggio imperativo.

4.6 Pascal

progettato da **N. Wirth** agli inizi degli anni '70, prende il nome dal filosofo **Blaise Pascal**, inventore della **pascalina**, strumento di calcolo precursore della calcolatrice moderna. Rappresenta una versione più semplice di ALGOL 60.

Linguaggio fortemente tipizzato, gestione dinamica della memoria e scoping statico.

Offre molti tipi di dati (anche custom) come array, record, file e set.

Supporta le funzioni.

4.7 Smalltalk

Introdotta negli anni '70 da **Alan Kay** allo Xerox Palo Alto Research Center per supportare nozioni di incapsulamento e information hiding.

Tutto è un oggetto tranne le variabili; la computazione avviene tramite uno scambio di messaggi.

Utilizza il garbage collector ed è dotato di GUI e IDE integrati.

4.8 PROLOG

PROLOG (PROgrammation en LOGique) fu introdotta da **Robert Kowalski** (teoria), **Marten Van Emden** (dimostrazione sperimentale) e implementata finalmente da **Alain Colmerauer** all'inizio degli anni '70.

Linguaggio dichiarativo

4.9 C++

Introdotta nei Bell Labs nel 1986 da **Bjarne Stroustrup**.

Gli oggetti sono una generalizzazione di "struct".

Estende **C** con le classi ed un migliore sistema di tipi.

Supporta concetti complessi come l'ereditarietà multipla, modelli / generici (template) e la gestione delle eccezioni.

4.10 Java

Introdotta alla SUN 1991-1995 dal gruppo di **James Gosling**, completamente orientata agli oggetti.

Ha garbage collection e quindi la gestione della memoria non è esplicita; non c'è ereditarietà multipla e neanche generics.

Si possono usare thread in maniera concorrente.

Presta molto attenzione alla portabilità dato il suo aspetto ibrido tra linguaggio compilato e interpretato.

4.11 Altri linguaggi popolari

- **Imperativi:** Modula, Oberon, Ada
- **Funzionali:** Miranda, Haskell, ML, Caml, OCaml, Scheme, Common LISP
- **Orientati agli oggetti:** Objective-C, Eiffel, Modula-3, Self, C#, CLOS
- **Logici:** Gödel, LDL, ACL2, Isabelle, HOL
- **Scripting:** (Text processing) Perl, Python; (Web programming) JavaScript, PHP
- **Elaborazione dati e database:** Cobol, SQL, 4GL, XQuery
- **Programmazione di sistemi:** PL/I, PL/M, BLISS
- **Applicazioni specializzate:** APL, Forth, Icon, Logo, SNOBOL4, GPSS, Visual Basic
- **Concorrenti:** Concurrent Pascal, Concurrent C, C*, SR, Occam, Erlang, Obliq

4.12 Come crescono i linguaggi?

Ci sono diversi fattori che fanno crescere un linguaggio, eccone diversi:

- **Killer App:** un linguaggio è la scelta gettonata per sviluppare app di successo (Java, PHP, R)
- **Esclusività delle piattaforme:** alcuni linguaggi si impongono perchè vincolati a un ecosistema (Swift e iOS).
- **Velocità dell'upgrade:** evoluzione rapida che mantiene la semplicità (Python)
- **Grande slogan:** fa la differenza (Java: write once, run everywhere)
- **Slow and steady:** crescita lenta ma stabile e duratura (C)
- **Sintassi chiara:** abbassa la barriera d'ingresso (Python)
- **Job market:** orienta le scelte (Java, JavaScript, Python)
- **Community:** il contributo degli appassionati tramite numerosissime librerie (JavaScript, Python)

5 Approccio scientifico

Per studiare i linguaggi di programmazione si attuerà un **metodo scientifico**:

1. **Fatti:** la realtà che va studiata (osservazioni, esperimenti, dati...)
2. **Modello:** la rappresentazione tramite leggi matematiche.
3. **Ragionamento:** analisi della realtà attraverso il modello.

Questo metodo viene applicato in tutti i campi della scienza e ricerca e lo applicheremo anche noi in questo modo:

1. **Fatti:** linguaggi di programmazione.
2. **Modello:** semantica.
3. **Ragionamento:** comprensione e studio di proprietà.

6 Lambda Calculus

6.1 Definizione

Il **lambda calcolo** è stato introdotto nel 1935 da **Church** e può essere considerato il primo linguaggio di programmazione funzionale e setta proprio una base per studiare tale paradigma.

6.2 Lambda espressioni

6.2.1 Segnatura e funzione identità

Il lambda calculus utilizza questa segnatura:

$$\lambda x.x$$

- λ = introduce la funzionale
- x = parametro formale (1); corpo (2) la cui valutazione applicata all'input dà il valore in output.
- $.$ = separa la variabile dal corpo

La legge di corrispondenza corrisponde a:

$$f(x) = x$$

Ovvero la funzione identità.

6.2.2 Applicazione

Per attualizzare questa segnatura si fa così e si chiama **applicazione**:

$$(\lambda x.x)z = z$$

Con "z" come **parametro attuale**.

6.2.3 Funzione costante

Vediamo adesso un esempio di **funzione costante**:

$$\lambda x.y$$

$$(\lambda x.y)z = y$$

L'output non dipende dall'input (parametro formale).

6.2.4 Funzioni annidate

Addirittura, esistono annidamenti di funzioni:

$$\lambda x. \lambda y. x$$

Questo aspetta due parametri in input e restituisce sempre il primo, corrisponde a:

$$f(x, y) = x$$

$$((\lambda x. \lambda y. x)z)w = (\lambda y. z)w = z$$

La prima funzione non ritorna un valore, bensì un'espressione (o una funzione) che può essere usata a sua volta.

La parte dell'espressione che si riduce si chiama **redex**, si effettuano le sostituzioni in ordine e si osserva come si evolve per determinare un risultato.

6.2.5 Ordine di semplificazione

Spesso si può scegliere un ordine di semplificazione:

- Partendo dalla funzione più esterna
- Partendo dalla funzione più interna

Vediamo un esempio:

$$(\lambda x. ((\lambda y. y)x))z$$

Risolvendo prima l'esterna:

$$(\lambda x. ((\lambda y. y)x))z = \lambda(y. y)z = z$$

Risolvendo prima l'interna:

$$(\lambda x. ((\lambda y. y)x))z = \lambda(x. x)z = z$$

6.2.6 Funzioni come parametro

Vediamo adesso un altro caso:

$$(\lambda f. fz)(\lambda x. x)$$

Questo usa un'intera funzione come parametro:

$$(\lambda f. fz)(\lambda x. x) = (\lambda x. x)z = z$$

6.2.7 Prolungamento di espressione

Le espressioni possono ricostruirsi:

$$(\lambda x. (xx))(\lambda y. y)$$

$$(\lambda x. (xx))(\lambda y. y) = (\lambda y. y)(\lambda y. y) = \lambda y. y$$

6.2.8 Parametri attuali numerici

Adesso vediamo un esempio con una somma:

$$((\lambda x.\lambda y.x + y)3)5$$

$$((\lambda x.\lambda y.x + y)3)5 = (\lambda y.3 + y)5 = 3 + 5 = 8$$

Basta effettuare un passaggio alla volta.

6.2.9 Ambiguità

Vediamo una situazione di ambiguità con 3 espressioni concatenate nella forma (e_1, e_2, e_3) :

$$(\lambda x.\lambda y.xy)(\lambda x.x)z$$

Per convenzione bisogna svolgere le prime due e lasciare la terza per dopo:

$$(\lambda x.\lambda y.xy)(\lambda x.x)z = (\lambda y.((\lambda x.x)y))z = \lambda(x.x)z = z$$

Questo conta anche per i parametri attuali:

$$(\lambda x.\lambda y.xy)zk$$

$$(\lambda x.\lambda y.xy)zk = (\lambda y.zy)k$$

Bisogna fare attenzione ad ambiguità tra parametri formali e attuali:

$$(\lambda x.\lambda y.xy)y$$

Se risolto, potrebbero esserci ambiguità, per questo conviene cambiare nome:

$$y = z$$

6.3 Convenzioni sintattiche

Per prima cosa, la **portata** (scope) del costruttore lambda si estende il più a destra possibile.

$$\lambda x.\lambda y.xy \text{ è uguale a } \lambda x.(\lambda y.(xy))$$

L'applicazione funzionale associa a **sinistra**:

$$e_1 e_2 e_3 \text{ diventa } (e_1 e_2) e_3$$

6.4 Variabili libere e legate

Una **variabile legate** (**bound**) a un λ sono quelle che fanno parte di una lambda espressione, ovvero una dichiarazione (parametri formali).

Le **variabili libere** (**unbound**) invece corrispondono al contrario, come i parametri attuali, le variabili non enunciate da un λ , queste sono definite dall'insieme:

$$FV$$

Per esempio, possiamo applicarlo ad un'espressione:

$$FV(x) = \{x\}$$

$$FV((\lambda x.x)z) = \{z\}$$

Un termine si dice **chiuso** se:

$$Fv(E) = \emptyset$$

6.5 Definizione sintattica

- $e ::= x$
 - $\lambda x.e$ (function declaration)
 - $e e$ (function call)

6.6 Alberi per lambda espressioni

Un altro tipo di rappresentazione sono gli alberi, fatti in questo modo:

- Il costruttore λ è la radice.
- Sul ramo sinistro si trova il parametro formale.
- Sul ramo destro si trova il corpo, il quale può ricominciare con un λ .

6.7 Alpha equivalenza

Due espressioni sono dette **alpha equivalenza** se differiscono solo per i nomi delle variabili legate. Più semplicemente, si potrebbe "rinominare" qualche parametro per rendere le due espressioni identiche, questa operazione si chiama **alpha-conversione**.

Questa tecnica può essere applicata anche a problemi di ambiguità legati alla similarità tra nomi di parametri formali e attuali.

Vediamo alcuni esempi:

Sostituzione negata, variabile legata

$$((\lambda x.yx)w)\{x := \lambda k.kx\} = (\lambda x.yx)w$$

Alpha equivalenza

$$((\lambda x.yx)w)\{y := \lambda k.kx\} \equiv_{\alpha} ((\lambda z.yz)w)\{y := \lambda k.kx\} = (\lambda z.\lambda k.kxz)w$$

Sostituzione semplice, diretta

$$((\lambda x.yx)w)\{w := \lambda k.kx\} = (\lambda x.yx)(\lambda k.kx)$$

6.8 Valutazione di lambda espressioni

La notazione è questa per la valutazione:

$$eval(espressione)$$

Semplicemente, la valutazione corrisponde a:

- Osservare se bisogna effettuare un'alpha-conversione per risolvere ambiguità.
- Sostituire i parametri formali coi parametri attuali.
- Svolgere la funzione (equivalente di effettuare una chiamata ad una funzione, per capire).

- Valutare il risultato e restituirlo.

La notazione per la sostituzione è questa:

$$e_1\{x := e_2\}$$

Nozione alternativa:

$$e_1\{e_2/x\}$$

La sostituzione può essere distribuita:

$$(e_1 e_2)\{x := e\} \equiv (e_1\{x := e\})(e_2\{x := e\})$$

Altri concetti importanti:

$$(\lambda y. e_1)\{x := e\} \equiv \lambda y. (e_1\{x := e\}), y \neq x \wedge y \notin FV(e)$$

ATTENZIONE: se nell'espressione con cui si sta effettuando la sostituzione (non la variabile da sostituire) contiene variabili con lo stesso nome di variabili legate bisogna prima effettuare l'alpha conversione per evitare ambiguità ed errori.

Infatti, la sostituzione si può effettuare solo sulle variabili non legate.

6.9 Beta reduction e beta equivalenza

La **beta reduction** consiste nella chiusura transitiva e riflessiva (stella di Finy).

Consiste semplicemente nell'applicazione dei parametri attuali per eseguire sostituzioni:

$$(\lambda x. xy)z$$

$$e_1\{x := e_2\}$$

$$(\lambda x. xy)z\{x := z\} \rightarrow zy$$

- $e_1 \rightarrow e_2$ se si ottiene e_2 da e_1 in **un passo** di riduzione.
- $e_1 \Longrightarrow e_2$ se si ottiene e_2 da e_1 in **zero o più passi** di riduzione.

Con $\Longrightarrow$ indichiamo la chiusura riflessiva e transitiva.

Una espressione e_1 è **beta-riducibile** se:

$$e_1 \Longrightarrow e_2$$

Due espressioni e_1 ed e_2 sono **beta equivalenti** ($e_1 \equiv_\beta e_2$) quando:

- Sono identiche (modulo alpha-conversione)
- $e_1 \Longrightarrow e_2$ oppure $e_2 \Longrightarrow e_1$
- $e_1 \Longrightarrow e$ e anche $e_2 \Longrightarrow e$

6.10 Non terminazione

Durante lo svolgimento di un'espressione si possono incontrare loop, questo concetto si chiama **non terminazione**.

Vediamo un esempio:

$$(\lambda x.(xx))(\lambda x.(xx))$$

Svolgendo l'espressione, si ricrea sempre.

In realtà, tali espressioni possono:

- Terminare in "alcuni casi" (facendo specifiche scelte di riduzione)
- Non terminare in "altri casi" (con scelte diverse)

6.11 Forma normale

Esistono delle procedure di valutazione delle espressioni.

Quando una lambda espressione non può essere più ridotta si dice che è in **forma normale beta**, ovvero quando non si può più applicare la beta riduzione.

Una lambda espressione ridotta in forma normale rappresenta quindi il **valore calcolato**, il vero e proprio risultato.

6.12 Teorema di Church Rosser

Il teorema afferma che il risultato non cambia al cambiare dell'ordine delle scelte da eseguire durante la semplificazione di un'espressione di lambda calcolo, in particolare:

Se esistono più percorsi di scelte, allora essi convergono necessariamente in una singola e comune espressione.

6.13 Funzioni di ordine superiore e currying

In lambda calculus le funzioni hanno un solo parametro, quindi per rappresentare le funzioni con più parametri è necessario effettuare un **annidamento** di λ .

Equivale a trasformare una funzione $f(X \times Y) \rightarrow Z$ in una funzione $g : X \rightarrow (Y \rightarrow Z)$. Questa trasformazione si chiama **currying**.

Vantaggi del currying:

- **Uniformità**: ogni funzione prende un solo argomento (più semplice)
- **Applicazione parziale**: possibilità di utilizzare subito funzioni specializzate senza costrutti aggiuntivi.
- **Eleganza e minimalità**: bastano astrazione e applicazione per esprimere qualsiasi espressione.

In pratica semplifica il tutto e rende il lambda calculus molto più potente.

6.14 Valutazione dell'applicazione di funzione

Ecco i passi:

1. **Primo passo:** valutazione dell'espressione che definisce la funzione.
2. **Secondo passo:** due casi:
 - Si può calcolare un parametro prima di applicarlo (**Valutazione eager, Call by value**)
 - Si può applicare un parametro prima di calcolarlo (**Valutazione lazy, Call by name**)
3. Valutazione del corpo della funzione

Haskell utilizza lo stile lazy, mentre **OCaml** utilizza lo stile eager. Ecco alcune differenze tra call by name e call by value:

- **call by value:** argomenti valutati una sola volta, potrebbe non terminare.
- **call by name:** argomenti valutati anche più volte o anche mai, termina sempre.

6.15 Implementazione della ricorsione

Per rappresentare la ricorsione si usa il **combinatore "Y"**, il quale è una funzione di ordine superiore.

Essa è una funzione speciale che, data una funzione generica F , costruisce un punto fisso di F .

- **Combinatore:** espressione che non ha variabili libere.
- **Punto fisso:** simile al concetto di caso base della ricorsione.

Vale questo:

$$YF \equiv_{\beta} F(YF)$$

Vediamo un esempio di implementazione della ricorsione con il concetto del fattoriale:

$$\begin{aligned} Y &= \lambda f. (\lambda x. f(xx)) (\lambda x. f(xx)) \\ G &= \lambda f. \lambda n. \text{if}(n = 0) \text{then} 1 \text{else} n \cdot f(n - 1) \\ F &= YG = \lambda f. (\lambda x. f(xx)) (\lambda x. f(xx)) G \\ &= (\lambda x. G(xx)) (\lambda x. G(xx)) = G((\lambda x. G(xx)) (\lambda x. G(xx))) \end{aligned}$$

Come si può notare si sta effettuando una ricorsione, al quale si potrà poi applicare un parametro attuale per calcolarne il risultato.

6.16 Variabili booleane

Ecco i valori principali in lambda calcolo dei valori booleani:

- **TRUE:** $\lambda t.\lambda f.t$
- **FALSE:** $\lambda t.\lambda f.f$

Vediamo adesso invece come funziona il **NOT**:

$$NOT = \lambda b.bTRUEFALSE$$

Se si applica TRUE o FALSE al NOT si ottiene il corrispettivo risultato.

Diamo un'occhiata a **AND** e **OR**:

$$AND = \lambda b.\lambda c.bcFALSE$$

$$OR = \lambda b.\lambda c.bTRUEc$$

Vediamo come si applica l'AND:

$$\begin{aligned} AND\ TRUE\ TRUE &= (\lambda b.\lambda c.bc\ FALSE)\ TRUE\ TRUE \\ &\rightarrow TRUE\ TRUE\ FALSE \\ &\rightarrow TRUE \end{aligned}$$

$$\begin{aligned} AND\ TRUE\ FALSE &= (\lambda b.\lambda c.bc\ FALSE)\ TRUE\ FALSE \\ &\rightarrow TRUE\ FALSE\ FALSE \\ &\rightarrow FALSE \end{aligned}$$

$$\begin{aligned} AND\ FALSE\ TRUE &= (\lambda b.\lambda c.bc\ FALSE)\ FALSE\ TRUE \\ &\rightarrow FALSE\ TRUE\ FALSE \\ &\rightarrow FALSE \end{aligned}$$

$$\begin{aligned} AND\ FALSE\ FALSE &= (\lambda b.\lambda c.bc\ FALSE)\ FALSE\ FALSE \\ &\rightarrow FALSE\ FALSE\ FALSE \\ &\rightarrow FALSE \end{aligned}$$

E adesso l'OR:

$$\begin{aligned} OR\ TRUE\ TRUE &= (\lambda b.\lambda c.b\ TRUE\ c)\ TRUE\ TRUE \\ &\rightarrow TRUE\ TRUE\ TRUE \\ &\rightarrow TRUE \end{aligned}$$

$$\begin{aligned} OR\ TRUE\ FALSE &= (\lambda b.\lambda c.b\ TRUE\ c)\ TRUE\ FALSE \\ &\rightarrow TRUE\ TRUE\ FALSE \\ &\rightarrow TRUE \end{aligned}$$

$$\begin{aligned} OR\ FALSE\ TRUE &= (\lambda b.\lambda c.b\ TRUE\ c)\ FALSE\ TRUE \\ &\rightarrow FALSE\ TRUE\ TRUE \\ &\rightarrow TRUE \end{aligned}$$

$$\begin{aligned} OR\ FALSE\ FALSE &= (\lambda b.\lambda c.b\ TRUE\ c)\ FALSE\ FALSE \\ &\rightarrow FALSE\ TRUE\ FALSE \\ &\rightarrow FALSE \end{aligned}$$

6.17 Condizionale

La struttura corrisponde a questo:

$$[espressione][then][else]$$

Ecco come viene rappresentata:

$$IF = \lambda c. \lambda then. \lambda else. c then else$$

Ecco un esempio con TRUE:

$$\begin{aligned} IF TRUE e_1 e_2 &= (\lambda c. \lambda then. \lambda else. c then else) TRUE e_1 e_2 \\ &= (\lambda then. \lambda else. TRUE then else) e_1 e_2 = (\lambda else. TRUE e_1 else) e_2 = TRUE e_1 e_2 \\ &= (\lambda t. \lambda f. t) e_1 e_2 \\ &= (\lambda f. e_1) e_2 \\ &= e_1 \end{aligned}$$

L'esempio è analogo su FALSE.

6.18 Numeri

Possiamo codificare i numeri naturali con una funzione che, data una funzione s (successore), viene applicata n volte ad una funzione z (lo 0), con n = numero desiderato.

$$\begin{aligned} C_0 &= \lambda s. \lambda z. z \\ C_1 &= \lambda s. \lambda z. sz \\ C_2 &= \lambda s. \lambda z. s(sz) \\ C_3 &= \lambda s. \lambda z. s(s(sz)) \\ &\dots \\ C_n &= \lambda s. \lambda z. s^n z \end{aligned}$$

Vediamo adesso come svolgere varie operazioni:

$$\begin{aligned} SUCC &= \lambda n. \lambda s. \lambda z. s(ns z) \\ PLUS &= \lambda m. \lambda n. \lambda s. \lambda z. ms(ns z) = \lambda m. \lambda n. m SUCC n \\ TIMES &= \lambda m. \lambda n. m(PLUS n) C_0 \end{aligned}$$

In "SUCC" la n segna quante volte svolgere s su z (quindi potrebbe anche essere s^n ?).

7 Regole di inferenza

7.1 Sintassi

$$\begin{aligned}
 E &= \\
 E = TRUE &= FALSE = NV = \text{if } E \text{ then } E \text{ else } E = \text{succ } E = \text{pred } E = \text{isZero } E \\
 V &= TRUE = FALSE = NV \\
 NV &= 0 = 1 = 2 = \dots
 \end{aligned}$$

7.2 Successore

$$\begin{aligned}
 &\frac{E \rightarrow E^I}{\text{succ}(E) \rightarrow \text{succ}(E^I)} \\
 &\frac{m = n + 1}{\text{succ}(n) \rightarrow m}
 \end{aligned}$$

7.3 Predecessore

$$\begin{aligned}
 &\frac{E \rightarrow E^I}{\text{pred}(E) \rightarrow \text{pred}(E^I)} \\
 &\frac{n > 0, m = n - 1}{\text{pred}(n) \rightarrow m}
 \end{aligned}$$

7.4 IsZero

$$\begin{aligned}
 &\frac{E \rightarrow E^I}{\text{isZero}(E) \rightarrow \text{isZero}(E^I)} \\
 &\frac{n = 0}{\text{isZero}(n) \rightarrow TRUE} \\
 &\frac{n > 0}{\text{isZero}(n) \rightarrow FALSE}
 \end{aligned}$$

7.5 If-Else

$$\begin{aligned}
 &\text{if } TRUE \text{ then } E_1 \text{ else } E_2 \rightarrow E_1 \\
 &\text{if } FALSE \text{ then } E_1 \text{ else } E_2 \rightarrow E_2 \\
 &\frac{E \rightarrow E^I}{\text{if } E \text{ then } E_1 \text{ else } E_2 \rightarrow \text{if } E^I \text{ then } E_1 \text{ else } E_2}
 \end{aligned}$$

8 Controllo dei tipi

Il controllo dei tipi nella grammatica corrisponde ad un'associazione binaria tra **espressione** e **tipo**.

$$E : T$$

8.1 Tipi

Ecco i tipi usati nel type checking:

- **Nat:** numeri naturali
- **Bool:** valori booleani

Type Checking dei valori sintattici:

$$false : Bool$$

$$true : Bool$$

$$NV : Nat$$

8.2 Well-typed e ill-typed

Un'espressione E è **well-typed** se è ben tipata. Il contrario (debolmente tipato) indica che E è un'espressione **ill-typed**.

8.3 Type Safety: Correttezza

La **correttezza** (type safety o soundness) del sistema di tipo è espressa formalmente da queste proprietà:

- **Progresso:** se $E : T$ allora E è un valore oppure $E \rightarrow E^I$ per una qualche espressione E^I .
- **Conservazione:** se $E : T$ e $E \rightarrow E^I$ allora $E^I : T$.

8.4 Controllo su successore

$$\frac{E : Nat}{succ(E) : Nat}$$

8.5 Controllo su predecessore

$$\frac{E : Nat}{pred(E) : Nat}$$

8.6 Controllo su isZero

$$\frac{E : Nat}{isZero(E) : Bool}$$

8.7 Controllo su if-else

$$\frac{E : Bool, E_1 : T, E_2 : T}{if\ E\ then\ E_1\ else\ E_2 : T}$$

9 Lambda Calculus Tipato

9.1 L'introduzione dei tipi

Nel linguaggio del lambda calcolo appena visto possono crearsi errori di semantica tipo:

$$IF0e_1e_2 = TRUE0$$

In questo caso si può applicare TRUE a 0 e creerebbe un risultato insensato.

Per questo sono necessarie delle **regole** e l'integrazione dei **tipi**.

Tipo: raggruppamento di valori in base al comportamento che vorremmo avessero. Descrive come il programma deve utilizzare il tipo stesso.

Per esempio:

$$3 + 2 : Int$$

Indica che ci si aspetta che il risultato sia un intero e che l'operatore binario "+" funzioni correttamente prendendo in input 2 interi.

9.2 Sistema di controllo dei tipi

In pratica andiamo ad applicare un vero e proprio **sistema dei tipi**, il quale è un **metodo sintattico, effettivo e strutturale** per dimostrare l'assenza di comportamenti anomali.

- **Sintattico:** la struttura sintattica guida il metodo di analisi del comportamento dei programmi.
- **Effettivo:** si può definire un **algoritmo** che controlla i vincoli sui tipi e implementarlo in un compilatore o interprete.
- **Strutturale:** i tipi assegnati alle componenti del programma è definito da quello delle sue sotto-componenti.

Il controllo dei tipi può essere:

- **Statico:** compile-time
- **Dinamico:** run-time

Il controllo dei tipi è fondamentale perchè è in grado di verificare alcune proprietà del programma:

- **Segretezza e autenticità** dei protocolli di sicurezza.
- **Assenza di deadlock** in sistemi concorrenti.

9.3 Sintassi dei tipi

Ecco la sintassi:

$$\begin{aligned}\tau &::= (\text{Tipi}) \\ \text{Bool} &(\text{Tipi dei booleani}) \\ \tau \rightarrow \tau &(\text{Tipo delle funzioni})\end{aligned}$$

Inoltre, vediamo le espressioni e i valori:

$$\begin{aligned}e &::= \\ &= x \\ &= \text{fun } x : \tau = e \\ &= e \ e \\ &= \text{Apply}(e, e) \\ &= \text{true} \\ &= \text{false} \\ &= n = 0, 1, 2, \dots \\ &= e \oplus e = \text{operazioni binarie } (+, -, *, /) \\ &= \text{if } e \text{ then } e \text{ else } e\end{aligned}$$

$$\begin{aligned}V &::= \\ &= \text{false} \\ &= \text{true} \quad = \text{fun } x : \tau = e\end{aligned}$$

Data questo nuovo alfabeto, vediamo la sintassi come viene ridefinita:

$$\begin{aligned}&\text{Apply}(\text{fun } x : \tau = e_1, v_2) \rightarrow e_1\{x := v_2\} \\ &\frac{e_1 \rightarrow e^I}{\text{Apply}(e_1, e_2) \rightarrow \text{Apply}(e^I, e_2)} \\ &\frac{e_2 \rightarrow e^I}{\text{Apply}(e_1, e_2) \rightarrow \text{Apply}(e_1, e^I)} \\ &\frac{e \rightarrow e_4}{\text{if } e_1 \text{ then } e_2 \text{ else } e_3 \rightarrow \text{if } e_4 \text{ then } e_2 \text{ else } e_3} \\ &\quad \text{if true then } e_2 \text{ else } e_3 \rightarrow e_2 \\ &\quad \text{if false then } e_2 \text{ else } e_3 \rightarrow e_3\end{aligned}$$

9.4 Ambiente di tipo

L'**ambiente di tipo** (Type Enviroment) è fondamentale per effettuare il type-checking sulle **variabili**.

Si indica con Γ ed è una **funzione** (di dominio finito) che associa nomi a tipi:

$$\Gamma = \{(x_1 : \tau_1), \dots, (x_k : \tau_k)\}$$

Per indicare la funzione:

$$\Gamma(x_i) : \tau_i$$

Se la funzione viene applicata su una variabile che non si trova nell'ambiente, la funzione restituisce **undefined**.

Il controllo effettivo di tipo si chiama **giudizio di tipo** e si indica così:

$$\Gamma \vdash e : \tau$$

Queste regole sono applicate dal compilatore tramite analisi statica.

Vediamo una regola di inferenza molto importante sul giudizio di tipo:

$$\frac{\Gamma(x) : \tau}{\Gamma \vdash x : \tau}$$

Vediamone un'altra per le funzioni:

$$\frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \text{fun } x : \tau_1 = e : \tau_1 \rightarrow \tau_2}$$

E ancora per la chiamata di funzione:

$$\frac{\Gamma \vdash e : \tau_1 \rightarrow \tau_2 \quad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash \text{Apply}(e_1, e_2) : \tau_2}$$

9.5 Substitution Lemma

Il lemma dice:

”I tipi sono preservati dall'operazione di sostituzione”

Se:

$$\begin{aligned} \Gamma, x : \tau_1 \vdash e : \tau \\ \Gamma \vdash e_1 : \tau_1 \end{aligned}$$

Allora:

$$\Gamma \vdash e\{x := e_1\} : \tau$$

9.6 Dichiarazioni locali

Stabiliamo che:

$$e ::= \text{let } x = e_1 \text{ in } e_2 : \tau_2$$

Significa che l'associazione ” $x = e_1$ ” vale in e_2 .

Regola di inferenza:

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \text{let } x = e_1 \text{ in } e_2 : \tau_2}$$

Questo significa che il valore della variabile locale rimane nel blocco e_2 e non sovrascrive il tipo ritornato da tale espressione (non esce).

9.7 Ricorsione

L'autoapplicazione non è tipabile:

$$\begin{aligned} e &::= \text{fix } e \\ e &\rightarrow e^I \\ \hline \text{fix } e &\rightarrow e^I \end{aligned}$$

$$\overline{\text{fix}(\text{fun } f : \tau = e) \rightarrow e[f := (\text{fix}(\text{fun } f := e : \tau))]}$$

10 OCaml

10.1 ML, un antenato

ML (**M**eta **L**anguage) fu proposto da Robin Milner nel 1972 e sviluppato negli anni '80.

Era un linguaggio fortemente tipato.

Caml era una versione

10.2 Come mai NON studiare OCaml (Ma invece si)

OCaml non è:

- Tra i linguaggi più popolari.
- Semplice.
- Il linguaggio del futuro.

Nonostante questo, è utile studiarlo perché molti linguaggi hanno preso concetti da OCaml e perché è un linguaggio che mostra una programmazione funzionale realizzata al meglio. Lo useremo per implementare in modo naturale semantica statica e dinamica, ma anche per realizzare **interpreti**.

10.3 Esecutore di OCaml

Per utilizzare OCaml useremo:

- `ocaml`: interprete standard eseguibile da terminale.
- `utop`: interprete con interfaccia avanzata.
- Interprete online.

L'esecutore si chiama anche **interactive toplevel** e funziona così:

- Esegue un REPL (Read-Eval-Print Loop).
- Compila e valuta immediatamente ogni espressione, fornendo il risultato e il suo tipo.
- Il risultato può essere legato ad un nome, ottenendo una **dichiarazione**.
- Un'espressione può usare risorse precedentemente definite (variabili, funzioni...)

Vediamo un esempio di dichiarazione:

```
In [ ]: let x = 12.0 ;;
```

Adesso un'espressione:

```
In [ ]: x + 5 ;;
```

11 Esercizi

11.1 Esercizio 1

Sottolineare le variabili libere nelle seguenti espressioni:

1. $(\lambda x.x)(\lambda y.xx)(\lambda y.y)y$
2. $(\lambda x.xk)((\lambda x.\lambda y.xy)(\lambda z.(\lambda x.z)y))$
3. $x(\lambda x.(\lambda y.yy)x)y$

Soluzione (non controllata):

1. $(\lambda x.x)(\lambda y.\underline{xx})(\lambda y.y)\underline{y}$
2. $(\lambda x.x\underline{k})((\lambda x.\lambda y.xy)(\lambda z.(\lambda x.z)\underline{y}))$
3. $\underline{x}(\lambda x.(\lambda y.yy)x)\underline{y}$

11.2 Esercizio 2

Date le seguenti λ -espressioni, individuare le sottoespressioni riducibili tramite la β -riduzione (i “redex”):

1. $(\lambda x.x)((\lambda x.x)(\lambda z.(\lambda x.x)z))$
2. $(\lambda x.x)(\lambda x.xx)(\lambda y.y)$
3. $x((\lambda x.(\lambda y.yy)x)y)$

Soluzione Soluzione non presente, complicata da rappresentare in LaTeX.

11.3 Esercizio 3

Calcolare il risultato delle seguenti sostituzioni (capture avoiding):

1. $(\lambda y.xy)\{x := y\}$
2. $(\lambda y.xy)\{x := \lambda y.y\}$
3. $(\lambda y.xy)\{x := \lambda x.xy\}$

Soluzione

1. $(\lambda y.xy)\{x := y\} \equiv_{\alpha} (\lambda z.xz)\{x := y\} = \lambda z.yz$
2. $(\lambda y.xy)\{x := \lambda y.y\} = \lambda y.(\lambda y.y)y$
3. $(\lambda y.xy)\{x := \lambda x.xy\} \equiv_{\alpha} (\lambda z.xz)\{x := \lambda x.xy\} = \lambda z.(\lambda x.xy)z$

11.4 Esercizio 4

Applicare la β -riduzione alle seguenti espressioni fino a raggiungere un'espressione in forma normale (cioè, non ulteriormente riducibile) o ad accorgersi che la β -riduzione non termina. Ad ogni passo, sottolineare il “redex” considerato per l'applicazione della β -riduzione. In caso di possibili soluzioni alternative, svolgere l'esercizio più volte esplorando tutte le possibilità.

1. $(\lambda x.\lambda y.xy)(\lambda z.zy)y$
2. $(\lambda x.\lambda y.xy)(\lambda x.yy)k$
3. $(\lambda x.(\lambda y.xy)(\lambda k.k))(xy)$
4. $(\lambda x.y)((\lambda y.yyy)(\lambda x.xxx))$
5. $(\lambda z.((\lambda x.\lambda y.xy)z)z)(\lambda z.zz)$

Soluzione

11.5 Esercizio 5

Ripetere l'esercizio 1.4 applicando le strategie di riduzione Call-by-value e Call-by-name.

Soluzione

11.6 Esercizio 6

Dimostrare la β -equivalenza delle seguenti coppie di λ -espressioni. La β -equivalenza vale per ogni coppia ed è richiesto di dimostrarlo. Nella dimostrazione, cercare di svolgere il numero minore possibile di passi di β -riduzione:

1. $(\lambda x.x)(\lambda y.y)z \equiv_{\beta} (\lambda x.(\lambda k.k))wz$
2. $(\lambda x.x)(\lambda y.y) \equiv_{\beta} (\lambda y.y)(\lambda x.x)$
3. $(\lambda x.\lambda y.xy)y \equiv_{\beta} (\lambda x.(\lambda x.x)(\lambda z.yz))k$

Soluzione